

# Conception d'Interface Web

## Responsive Web Design

Simon BERNARD

[simon.bernard@univ-rouen.fr](mailto:simon.bernard@univ-rouen.fr)



# Responsive vs Adaptive

Concevoir une UI web adaptée à tous les écrans:

## 1. *Adaptive Design*

- Plusieurs versions de la page pour différentes tailles d'écran.
- Le système identifie le type d'écran et charge la version appropriée (côté serveur).
- Basé sur des points de ruptures qui définissent les différentes versions.

## 2. *Responsive Design*

- Une seule version de la page qui s'adapte à la taille de l'écran (côté client).
- Utilise des unités/grilles/médias flexibles.
- Basé sur des outils CSS qui appliquent des styles différents selon l'écran.

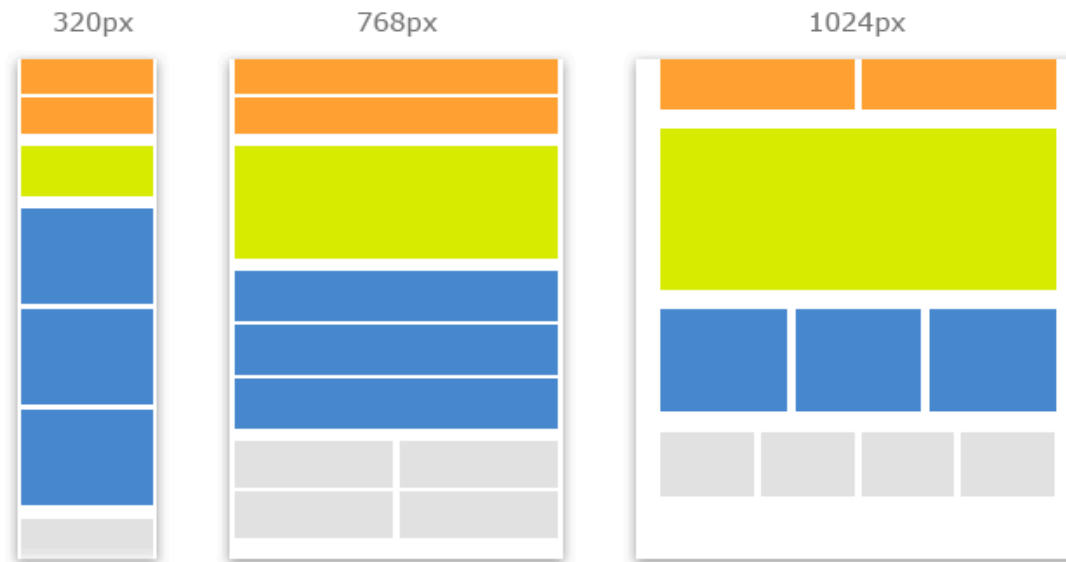
(démonstration: [http://g-mops.net/epica\\_saitama/epica\\_layout/index\\_adaptive.html](http://g-mops.net/epica_saitama/epica_layout/index_adaptive.html))

# Viewport

## Viewport

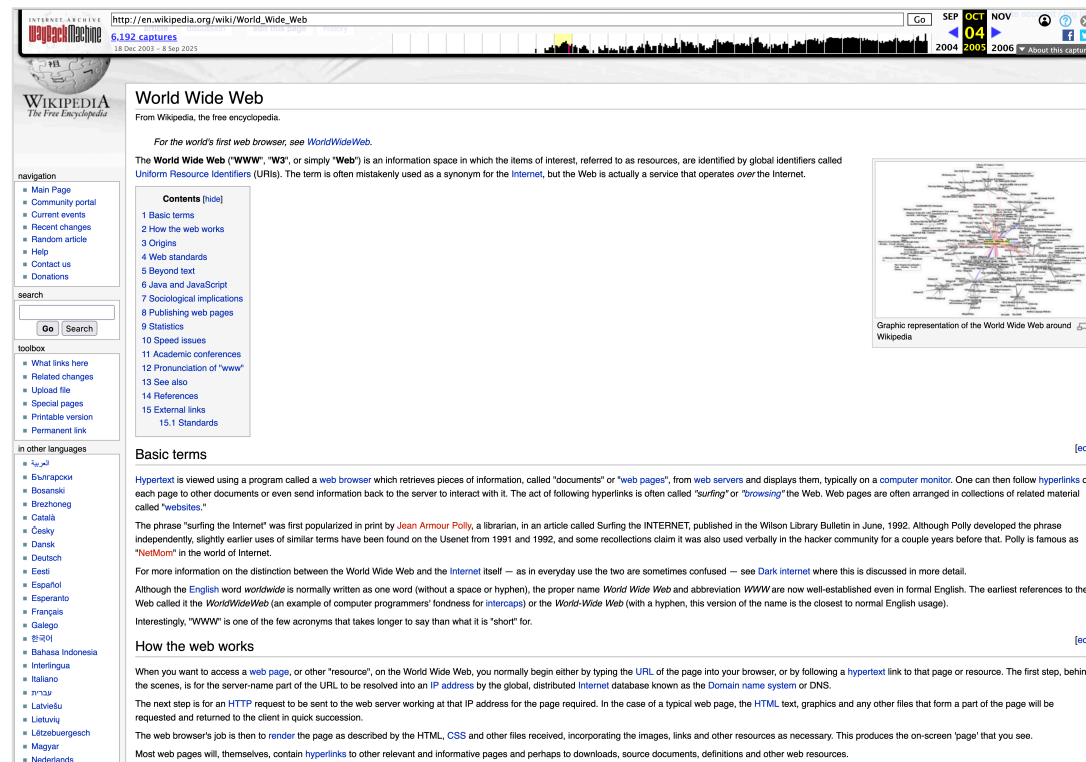
Zone dans laquelle est affiché un contenu. Désigne le plus souvent l'intérieur de la fenêtre d'un navigateur web mais ce peut aussi être un webview (e.g. dans les applications mobiles).

- Responsive Web Design = s'adapter automatiquement à la largeur du viewport
- Le navigateur (ou webview) ajuste le rendu en s'appuyant sur CSS



# Viewport

- HTML s'adapte déjà à la taille du viewport (cf. flux normal)
- Souvent insuffisant pour les écrans très larges ou très étroits
- Exemple: *Fluid layout* des années 2000



## Balise <meta name="viewport" ...>

- Les navigateurs mobiles définissent la largeur du viewport à 980 pixels (et non à la largeur du terminal)
- Les sites non optimisés pour le mobile s'affichent comme des versions dé-zoomées du site desktop
- Pour corriger ce comportement:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

- Définit la largeur du viewport à la largeur de l'appareil
- Annule le "dé-zoomage"
- À copier-coller sur toutes vos pages *responsive*

# Responsive Web Design

- RWD = techniques pour adapter automatiquement la mise en page aux dimensions du viewport
- Initialement (2010):
  1. *Media Queries*: styles alternatifs en fonction des caractéristiques viewport
  2. *Grilles flexibles*: agencement en grille dans laquelle les éléments sont positionnés
  3. *Images flexibles*: dimensions des images qui s'ajustent à l'espace disponible
- Aujourd'hui, bien plus: *flexbox/grid*, orientation, taille du pointeur, *viewport segments*, etc.

# Media Queries

---

# Media queries

- Mécanisme CSS pour appliquer des styles conditionnels
- Exemple: styles différents pour l'impression:

```
nav, footer {  
    background-color: lightgray;  
}  
@media print {  
    nav, footer { display: none; }  
}
```

ou

```
<link rel="stylesheet" media="screen" href="screen.css" type="text/css">  
<link rel="stylesheet" media="print" href="print.css" type="text/css">
```



# Types de media possibles

- **screen**: pour les écrans
  - **print**: pour l'impression
  - **speech**: pour les synthétiseurs vocaux
  - **all**: pour tous les appareils (défaut)
- 
- **projection**: vidéo-projecteur (*deprecated*)
  - **tv**: téléviseurs (*deprecated*)
  - **handheld**: appareils portables (*deprecated*)
  - **braille**: afficheurs braille (*deprecated*)
  - **embossed**: imprimantes braille (*deprecated*)
  - **tty**: terminaux à caractères (*deprecated*)

# Media features

- Pour plus de précision, on peut ajouter des conditions sur les caractéristiques des appareils

```
@media type and (feature) {  
    /* styles */  
}
```

ou

```
<link rel="stylesheet" href="specific.css" media="type and (feature)">
```

- Exemple:

```
@media all and (orientation: portrait) {. /* ici, "all and" est optionnel */  
    aside { display: none; }  
}
```

# Media features les plus utilisés

- **min-width, max-width**: pour cibler la largeur minimale ou maximale du viewport

```
@media screen and (min-width: 600px) {  
    /* Styles pour les écrans d'au moins 600px de large */  
}  
@media screen and (max-width: 599px) {  
    /* Styles pour les écrans de moins de 599px de large */  
}
```

- Peut aussi s'écrire avec la propriété **width**:

```
@media screen and (width >= 600px) {  
    /* Styles pour les écrans de 600px de large */  
}
```

- Idem avec **height**

# Media features les plus utilisés

- Le conditions peuvent être combinées:

```
@media (min-width: 600px) and (max-width: 900px) {  
    /* Styles pour les écrans entre 600px et 900px de large */  
}
```

ou

```
@media (600px <= width <= 900px) {  
    /* Styles pour les écrans entre 600px et 900px de large */  
}
```

- Autre exemple:

```
@media not ((width >= 30em) or (orientation: landscape)) {  
    /* ces styles ne seront pas appliqués si le viewport est plus large que 30em, ou si l'orientation est paysage */  
}
```

# Media features les plus utilisés

- Si l'utilisateur dispose d'un curseur de souris:

```
@media screen and (hover: hover) {  
  /* styles appliqués si l'utilisateur peut survoler les éléments avec une souris */  
}
```

- Teste la grosseur du pointeur

```
@media screen and (pointer: coarse) { /* autre valeur: fine ou none */  
  /* le pointeur est grossier (doigt) */  
}
```

- Rapport d'aspect:

```
@media screen and (min-aspect-ratio: 4/3) {  
  /* écran de ratio 4/3 = 1.333 et plus */  
}
```

- Résolution (DPI):

```
@media screen and (min-resolution: 300dpi) {  
  /* écran de résolution 300dpi et plus */  
}
```

# Points de ruptures

- Souvent utilisés pour définir des règles spécifiques à des points de rupture (breakpoints)
- Style génériques en premier, puis media queries pour les règles spécifiques
- Cibler les viewport les plus petits en premiers:

```
/* styles génériques */  
@media screen and (min-width: 576px) { ... }  
@media screen and (min-width: 768px) { ... }  
@media screen and (min-width: 992px) { ... }  
@media screen and (min-width: 1200px) { ... }  
@media screen and (min-width: 1400px) { ... }
```

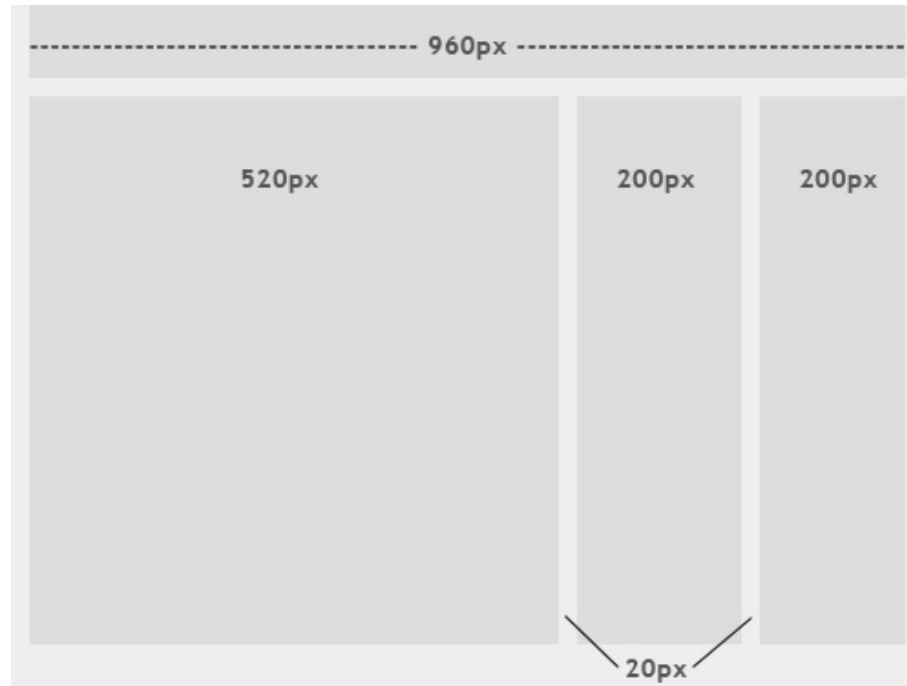
- *Desktop-first* ou *mobile-first*?

# Grilles flexibles

---

# Flexible layout

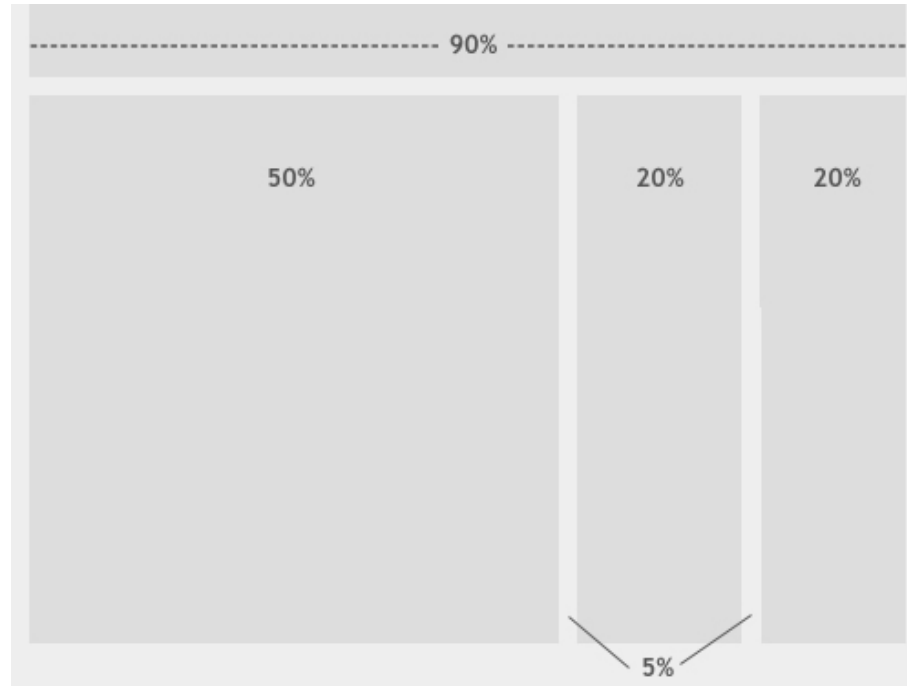
*Fixed layout*: la mise en page est basée sur des boîtes (cf. *box model* du chapitre précédent) de dimensions fixes, i.e. en pixels





# Flexible layout

*Flexible layout* (ou *Fluid* ou *Liquid*): les boîtes ont des dimensions et positions spécifiées dans une unité de mesure adaptative (e.g. pourcentage)



# Unité de mesure flexible

## ➤ % : pourcentage

```
.container {  
  width: 100%;    /* toute la largeur du conteneur */  
  max-width: 1200px; /* pas plus large que 1200px */  
}  
nav {  
  width: 87.5%; /* 87.5% de la largeur du conteneur */  
}
```

## ➤ em: unité relative à la taille de la police du conteneur

```
/* si police du conteneur de 16px */  
h1 {  
  font-size: 1.5em; /* taille de 16 x 1.5 = 24 px */  
}
```

# Unité de mesure flexible

- **rem**: unité relative à la taille de la police de l'élément racine (**<html>**)
- **vw**: unité relative à la largeur de la fenêtre d'affichage (viewport width)
- **vh**: unité relative à la hauteur de la fenêtre d'affichage (viewport height)
- **vmin**: unité relative à la plus petite dimension de la fenêtre d'affichage (viewport minimum)
- **vmax**: unité relative à la plus grande dimension de la fenêtre d'affichage (viewport maximum)

Convertisseur px/em: [https://www.w3schools.com/cssref/css\\_pxtoemconversion.php](https://www.w3schools.com/cssref/css_pxtoemconversion.php)

# Exemple

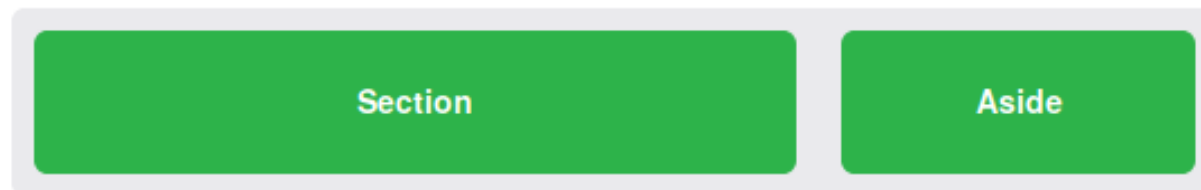
Agencement en deux colonnes, avec un conteneur incluant une section et un encadré côte à côte

```
<!-- en HTML -->
<body>
  <main class="container">
    <section> ... </section>
    <aside> ... </aside>
  </main>
</body>
```

# Exemple

Agencement en deux colonnes, avec un conteneur incluant une section et un encadré côte à côte

```
/* Fixed layout en CSS */  
.container {  
    width: 538px;  
}  
section, aside {  
    margin: 10px;  
}  
section {  
    float: left;  
    width: 340px;  
}  
aside {  
    float: right;  
    width: 158px;  
}
```



## Exemple

Pour calculer les dimensions en %:  $(\text{largeur élément} \div \text{largeur conteneur}) \times 100$

```
/* Fluid layout en CSS */
.container {
  width: 538px;
}
section, aside {
  margin: 1.858736059%; /* 10px / 538px = .018587361 */
}
section {
  float: left;
  width: 63.197026%; /* 340px / 538px = .63197026 */
}
aside {
  float: right;
  width: 29.3680297%; /* 158px / 538px = .293680297 */
}
```

# Exemple

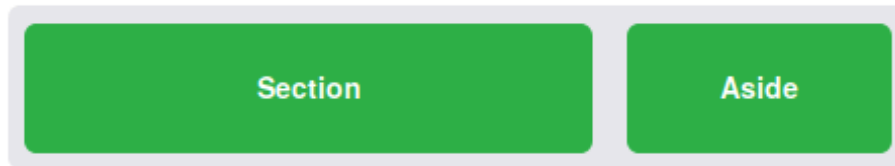
100% Wide Container



75% Wide Container



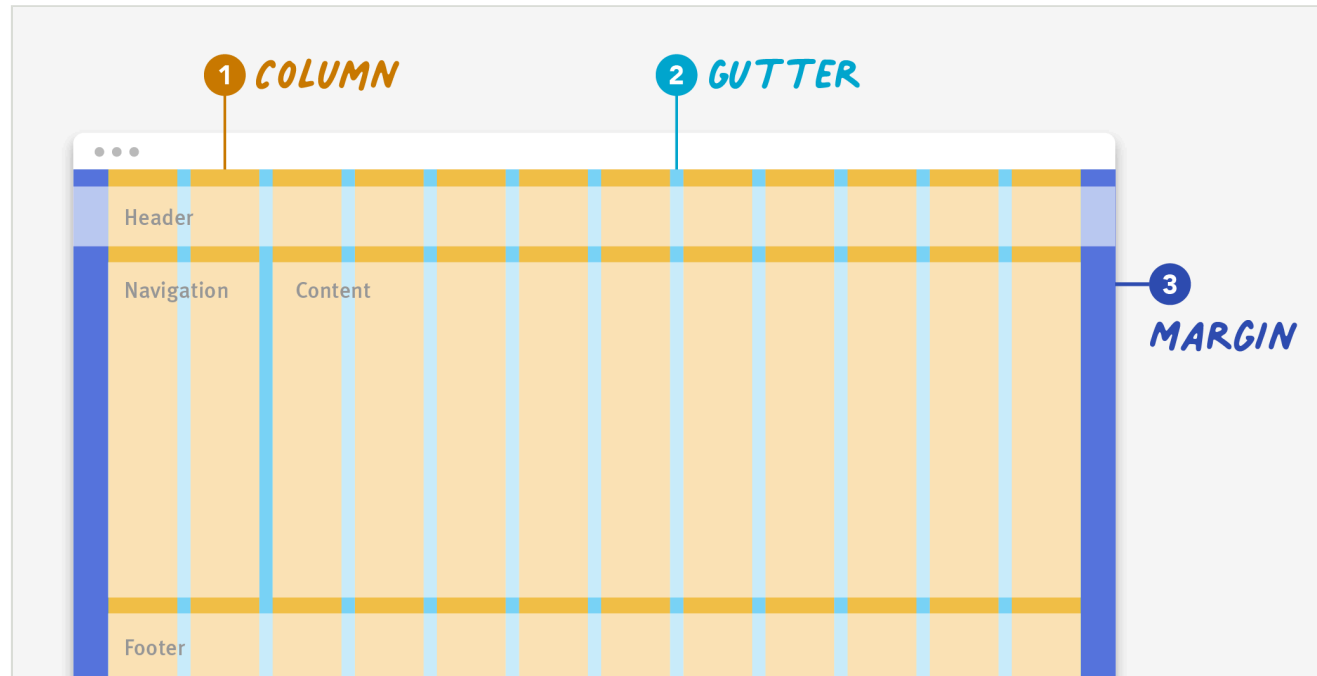
50% Wide Container



Exemple tiré de <https://learn.shayhowe.com/advanced-html-css/responsive-web-design/>

# Mise en page en grille flexible

- Basé sur ce principe, le plus courant est de s'appuyer sur une mise en page en grille flexible
- La largeur du viewport est divisé en colonnes de même largeur





# Mise en page en grille flexible

- La largeur d'un élément est définie en "nombre de colonnes"
- Classes CSS pré-définies pour rendre les éléments flexibles

```
.col-1 {width: 8.33%;}  
.col-2 {width: 16.66%;}  
.col-3 {width: 25%;}  
.col-4 {width: 33.33%;}  
.col-5 {width: 41.66%;}  
.col-6 {width: 50%;}  
.col-7 {width: 58.33%;}  
.col-8 {width: 66.66%;}  
.col-9 {width: 75%;}  
.col-10 {width: 83.33%;}  
.col-11 {width: 91.66%;}  
.col-12 {width: 100%;}
```

# Mise en page en grille flexible

- La largeur réelle comprend les espacements et bordures:

$\text{width} + \text{margin} + \text{padding} + \text{border} = \text{largeur réelle}$

- Compromet la subdivision en colonnes
- Pour corriger: `box-sizing: border-box;`. Les `padding` et `border` sont inclus dans `width`
- Pas le cas des marges qui doivent être mises à zéro ou prises en compte pour le découpage en colonnes
- Problème si répandus que beaucoup de développeurs utilisent:

```
* {  
  box-sizing: border-box;  
}
```

# Mise en page en grille flexible

- Les éléments doivent être flottants

```
[class*="col-"] {  
  float: left;  
}
```

- Chaque "ligne" est définie par un **div**

```
<div class="row">  
  <div class="col-3">...</div> <!-- 25% -->  
  <div class="col-9">...</div> <!-- 75% -->  
</div>
```

- Pour le retour à la ligne en dessous de chaque ligne

```
.row::after {  
  content: " ";  
  clear: both;  
}
```

# Principe mise en oeuvre dans les frameworks CSS (e.g. [Bootstrap](#))

There's a newer version of Bootstrap!

[Home](#) **[Docs](#)** [Examples](#) [Icons](#) [Blog](#)

[Download](#)

Search docs... Ctrl + / v5.0 ▾

- [Getting started](#)
- [Customize](#)
- ▼ **[Layout](#)**
  - [Breakpoints](#)
  - [Containers](#)
  - [Grid](#)**
    - [Columns](#)
    - [Gutters](#)
    - [Utilities](#)

## Grid system

[View on GitHub](#)

Use our powerful mobile-first flexbox grid to build layouts of all shapes and sizes thanks to a twelve column system, six default responsive tiers, Sass variables and mixins, and dozens of predefined classes.

# CSS Flexbox et Grid

---

# Nouveaux modules CSS d'agencements

- Introduit en 2017
- Permettent de créer des mises en page flexibles sans avoir à recourir aux flottants
- Deux modules:
  - **Flexbox** (Flexible Box Layout): pour agencer des éléments dans une seule dimension (ligne ou colonne)
  - **Grid** (CSS Grid Layout): pour agencer des éléments dans deux dimensions (lignes et colonnes)
- **Grid** pour les mises en pages "macro", **Flexbox** pour les mises en pages "micro"
- *grid system* de Bootstrap maintenant basé sur **Flexbox**

# Nouveaux modules CSS d'agencements

- Principe général: plusieurs propriétés CSS pour contrôler un conteneur et ses items
- Le conteneur a la capacité de modifier les dimensions et l'agencement de ses items
- Les items peuvent aussi avoir des propriétés spécifiques pour s'adapter au conteneur
- On définit le conteneur via la propriété **display**

```
.container {  
  display: flex; /* ou grid */  
}
```

- Les items ont des propriétés spécifiques pour s'adapter au conteneur

```
.item {  
  flex-grow: 1; /* pour flexbox */  
  grid-column: span 2; /* pour grid */  
}
```

# Flexbox container

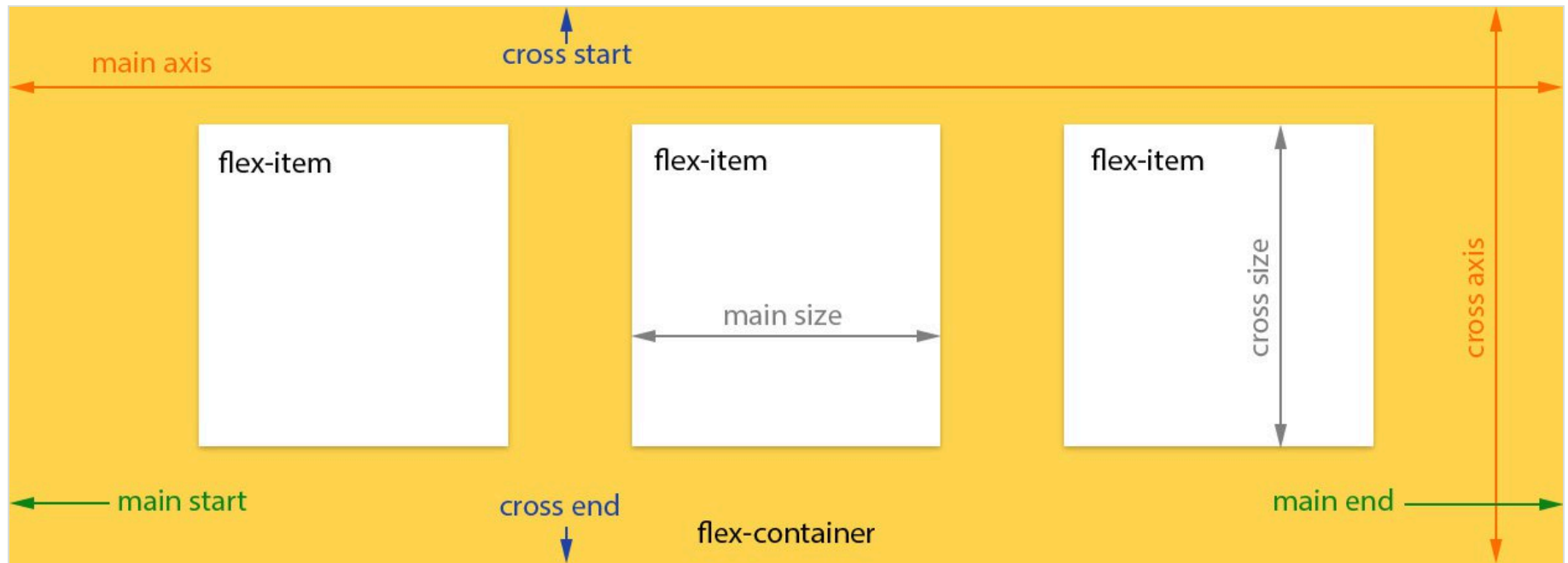
- Agencement en une dimension (ligne ou colonne)
- 4 directions possibles:

```
.container {  
  display: flex;  
  flex-direction: row; /* row | row-reverse | column | column-reverse */  
}
```

1. ligne (par défaut, de gauche à droite)
  2. ligne inversée (de droite à gauche)
  3. colonne (de haut en bas)
  4. colonne inversée (de bas en haut)
- Le reste du positionnement/dimensionnement se fait indépendamment



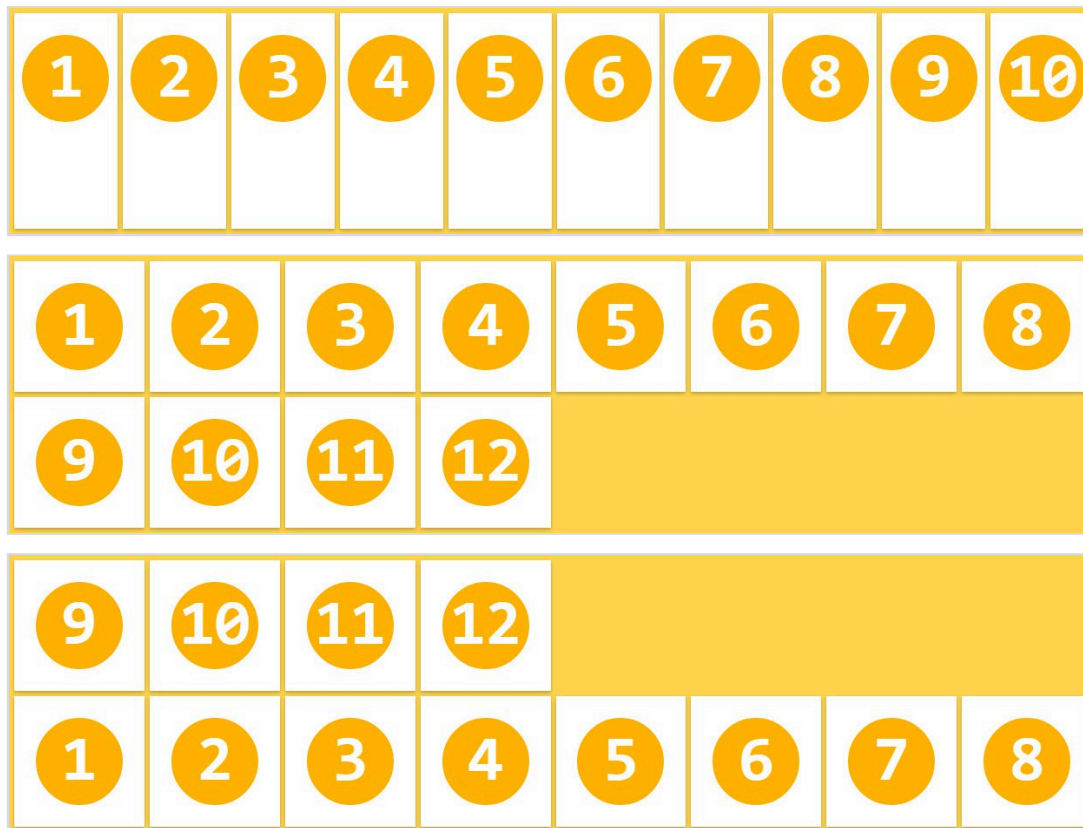
# Flexbox model



- 2 axes: principal (main) et transversal (cross)
- 2 extrémités: début (start) et fin (end)

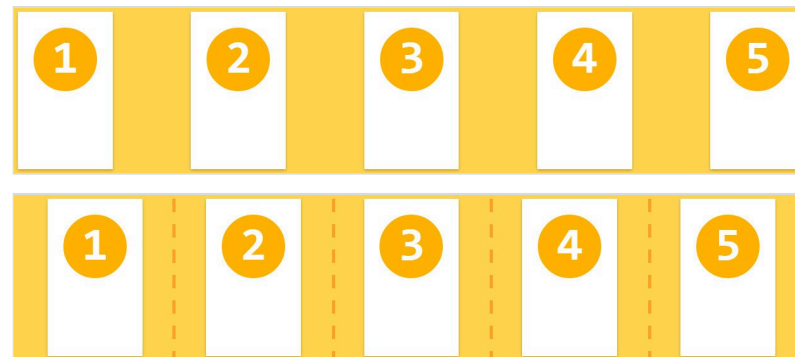
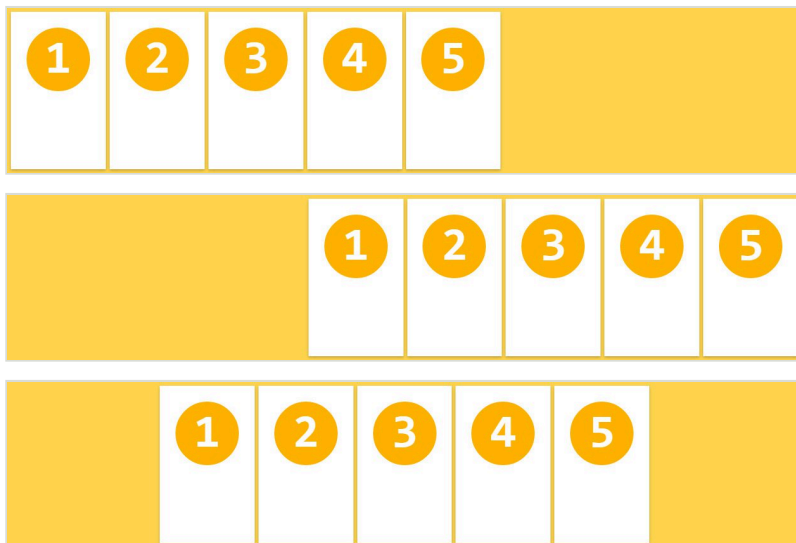
## Propriétés du conteneur

- flex-wrap: nowrap | wrap | wrap-reverse;
- Si les items sont distribués sur une ou plusieurs lignes



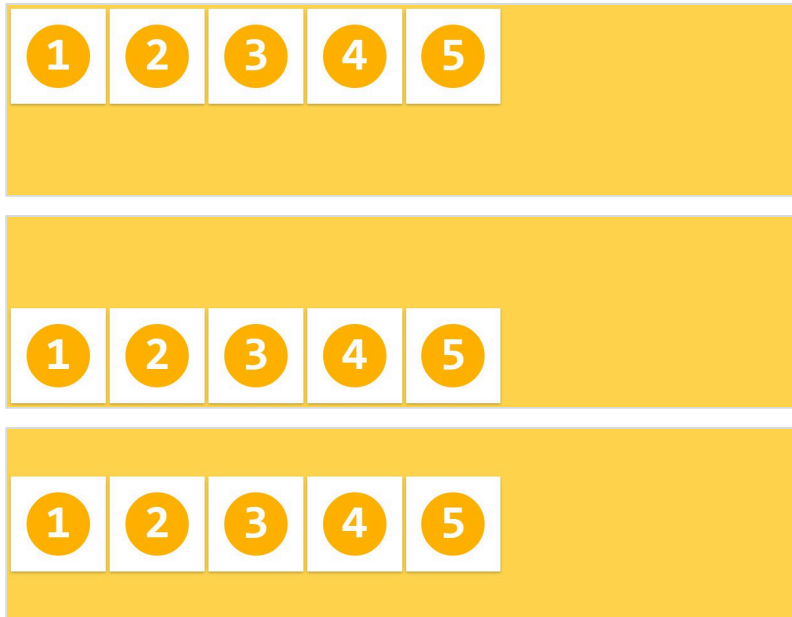
# Propriétés du conteneur

- `justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly;`
- Comment distribuer l'espace autour des items selon l'axe principal



# Propriétés du conteneur

- `align-items: flex-start | flex-end | center | baseline | stretch;`
- Comment aligner les items selon l'axe transversal



# Propriétés du conteneur

- align-content: flex-start | flex-end | center | space-between | space-around | stretch;
- Comment distribuer l'espace entre les lignes/colonnes selon l'axe transversal

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 |    |    |    |    |

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 |    |    |    |    |

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 |    |    |    |    |

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 |    |    |    |    |

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 |    |    |    |    |

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 |    |    |    |    |

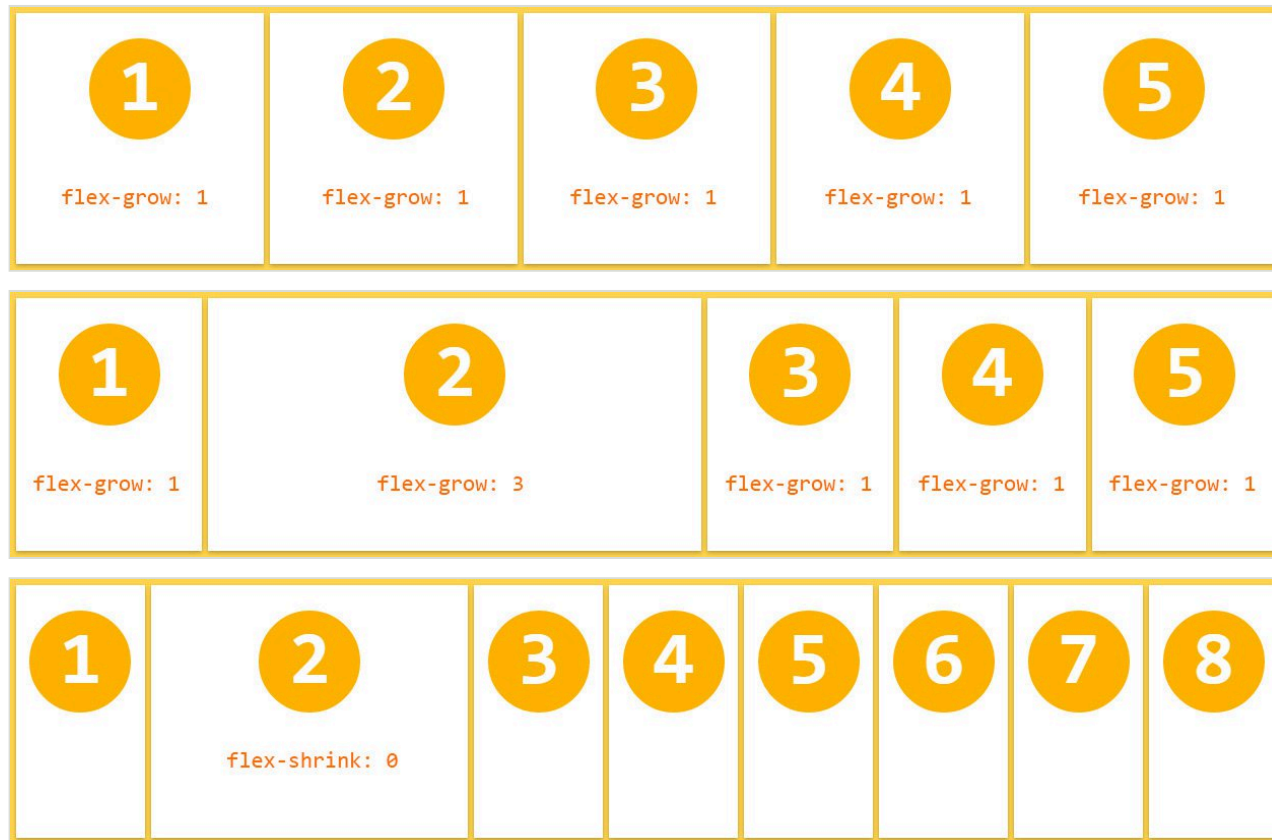
## Propriétés des items

- `order: <entier>;`
- La position de l'item. Par défaut, l'ordre est celui dans le HTML



# Propriétés des items

- `flex-grow: <entier>; -- flex-shrink: <entier>;`
- Facteurs d'agrandissements/rétrécissement des items, quand le conteneur est redimensionné



# Propriétés des items

➤ et d'autres...

**Flexbox Properties Demonstration**  
A Fork by Dimitar

**HTML**

```
1 <!-- tell angular this is flexboxDemoApp -->
2 <html lang="en" ng-app="flexboxDemoApp">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-
6     scale=1">
7   <title>CSS3 Flexbox Playground</title>
8   <!-- AngularJS Material CSS -->
9   <link rel="stylesheet" href="https://ajax.googleapis.com
10     /ajax/libs/angular_material/0.8.3/angular-material.min.css">
11   <!-- include our compiled CSS -->
12   <link href="css/style.css" rel="stylesheet">
13 </head>
14 <!-- this will be the only controller AppCtrl -->
15 <body ng-controller="AppCtrl">
```

**CSS (scss)**

```
1 @import "compass/css3";
2
3 @import "compass";
4
5 // Flex container
6 .flexbox-parent {
7   @include display-flex;
8   background: #FFD54F;
9   overflow: hidden;
10   width: 100%;
11   padding: 30px;
12   min-height: 550px;
13   max-height: 1000px;
14 }
15 // Flex items
```

**JS**

```
1 // Declare the flexboxDemoApp module and its dependency 'ngMaterial'
2 var app = angular.module('flexboxDemoApp', ['ngMaterial']);
3 // Declare the AppCtrl controller
4 app
5   .config(['$mdThemingProvider', function ($mdThemingProvider) {
6     $mdThemingProvider
7       .theme('default')
8       .accentPalette('amber', {
9         default: '700'
10       });
11   }])
12   .controller('AppCtrl', ['$scope', function ($scope) {
13     $scope.parent = {
14       flexDirection: 'row',
15       flexWrap: 'nowrap',
```

**Children Width**  
width: 12%

**Parent Flex Properties – flex container**

**flex-direction**

- ☐ row
- ☒ row-reverse
- ☐ column
- ☐ column-reverse

**flex-wrap**

- ☐ nowrap
- ☒ wrap
- ☐ wrap-reverse

**justify-content**

- ☐ flex-start
- ☐ flex-end
- ☒ center
- ☐ space-between
- ☐ space-around

**align-items**

- ☒ stretch
- ☐ flex-start
- ☐ flex-end
- ☐ center
- ☐ baseline

**align-content**

- ☒ stretch
- ☐ flex-start
- ☐ flex-end
- ☐ center
- ☐ space-between
- ☐ space-around

\* The default property values are highlighted.

**Children Flex Properties – flex items**

The children flex properties can be applied at child level, separate for each child. See the results section and change some of their properties. Hover with the mouse pointer or touch the fields to see the property name.

**Result**

ADD CHILD

| 5    | 4    | 3    | 2    | 1    |
|------|------|------|------|------|
| 0    | 0    | 0    | 0    | 0    |
| 0    | 0    | 0    | 0    | 0    |
| 1    | 1    | 1    | 1    | 1    |
| auto | auto | auto | auto | auto |
| auto | auto | auto | auto | auto |

Console Assets Comments Shortcuts

Share Export Embed

Tous les détails: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>



# Module Grid

- Agencement en tableau 2D

```
.container {  
  display: grid;  
  grid-template-rows: auto auto auto; /* 3 lignes de hauteur automatique */  
  grid-template-columns: auto auto auto auto; /* 4 colonnes de largeur automatique */  
  grid-gap: 10px; /* espacement de 10px entre les lignes et colonnes */  
}
```

- Les items sont placés dans la grille

```
.item {  
  grid-column: 3 / span 2; /* commence à la colonne 3 et s'étend sur 2 colonnes */  
  grid-row: 3 / 4; /* commence à la 3ème ligne et finit à la 4ème */  
}
```

- Plus complexe mais TRÈS puissant

Tous les détails: <https://css-tricks.com/snippets/css/complete-guide-grid/>

# Images flexibles

---

# Image flexible

- Image dont les dimensions s'ajustent en fonction du *viewport*
- Par défaut, une image est affichée avec ses dimensions d'origine
- Image flexible = CSS pour spécifier les dimensions avec une unité flexible, de préférence pourcentage

```
img {  
  width: 100%;      /* toute la largeur de son conteneur */  
  height: auto;     /* hauteur ajustée automatiquement */  
}
```

- Problème: si l'image est originellement moins large que le conteneur?

```
img {  
  max-width: 100%; /* pas plus large que son conteneur */  
  height: auto;    /* hauteur ajustée automatiquement */  
}
```

- Si l'image est plus large que le conteneur, elle est redimensionnée
- Si l'image est plus petite que le conteneur, elle garde ses dimensions d'origine
- cf. [démonstration](#)

# Image flexible

2 types de problèmes persistent:

**1.** Image fortement "dézoomée" sur un écran très étroit

- Très peu visible si image initialement très large
- Bande passante gaspillée

**2.** Image trop petite pour un écran très large

- Très peu visible si image initialement de faible résolution
- Si zoomé, forte pixelisation

- Les *media queries* peuvent solutionner une partie de ces problèmes mais pas tous
- Les SVG également mais pas adaptés aux images très détaillées (photos)
- CSS fournit également quelques outils intéressants mais pas suffisants (e.g. [démonstration MDN](#))

## Attributs `srcset` et `sizes`

- HTML5 fournit des attributs pour l'élément `<img>` pour adapter l'image à la résolution
- `srcset`: liste d'images alternatives avec leurs largeurs (en pixels)

```

```

- Le navigateur choisit l'image la plus adaptée en fonction de la largeur effective du conteneur
- Le descripteur `Xw` indique la largeur de l'image (et non du *viewport*)
- Si le navigateur ne prend pas en charge `srcset`, il utilise l'image spécifiée dans l'attribut `src`

## Attributs `srcset` et `sizes`

- `sizes`: liste de conditions sur la largeur du conteneur et la largeur d'image dans la page

```

```

- Largeurs spécifiées avec n'importe quelle unité de mesure CSS, sauf %
- Le navigateur applique la largeur de la première condition "vraie", la suite est ignorée
- La dernière largeur doit être la largeur par défaut

Remarque: c'est normalement CSS qui gère les dimensions mais le chargement des images débute en général avant que les traitements CSS ne soient terminés.

# L'élément source

- Fonctionne pour tous types de médias: **picture**, **video**, **audio**
- Utilisation de l'élément **<source>** pour spécifier des sources alternatives

```
<picture>
  <source media="(max-width: 600px)" srcset="image-cropped.jpg">
  <source media="(max-width: 1200px)" srcset="image-full.jpg">
  
</picture>
```

```
<video controls>
  <source src="foo.webm" type="video/webm">
  <source src="foo.ogv" type="video/ogg" />
  <source src="foo.mov" type="video/quicktime" />
  I'm sorry; your browser doesn't support HTML video.
</video>
```

- Le navigateur choisit la source la plus adaptée en fonction des conditions
- **img@srcset** et **picture+source** font la même chose

## En résumé...

- Responsive Web Design = un design pour tous les écrans
- Media queries pour appliquer des styles spécifiques selon les caractéristiques du dispositif
- Flexbox et Grid CSS pour des agencements flexibles
- Mise en oeuvre dans des grilles flexibles pour la structuration "macro"
- De nombreux frameworks CSS le font très bien pour nous, Bootstrap étant le plus répandu
- Images, vidéos, audios flexibles pour l'optimisation des performances et de l'expérience utilisateur